

Architecture fonctionnelle : de la user story au code (route → endpoint → DTO)

Le tableau « Carte fonctionnelle » de [DESIGN.md](#) relie chaque user story (US) à une **route Angular**, des **composants**, un **endpoint d'API** et des **DTO** (Data Transfer Objects). C'est la table de correspondance entre le besoin métier et le code des deux côtés : une US n'est « finie » que si toute sa ligne existe, côté front comme côté back.

Lire une ligne de la carte

Colonne	Question à laquelle elle répond	Où c'est dans le code
US	Quel besoin utilisateur ?	commentaires Javadoc / JSDoc (<code>US03</code> , <code>US02</code> ...)
Route Angular	Quelle URL du navigateur ?	<code>frontend/src/app/app.routes.common.ts</code>
Composants	Quel écran et quels morceaux d'UI ?	<code>features/*</code> et <code>shared/components/*</code>
Endpoint API	Quel appel HTTP vers le serveur ?	<code>*Controller.java</code>
DTO	Quelle forme de données échangée ?	<code>*/dto/*.java</code> (back) et <code>core/*/*.model.ts</code> (front)
Statut	Où en est-on ?	☒ fait, ☒ partiel, ☐ non traité

Une US = une ligne, mais une ligne peut **réutiliser** une autre : US09 (mot de passe fichier) n'ajoute ni route ni endpoint, elle ajoute un champ optionnel à ceux d'US01 et d'US02.

Côté front : route → composant → service

Les routes sont déclarées dans [app.routes.common.ts](#) :

```
{ path: 'd/:token', loadChildren: () => import('./features/download/download.routes') },
{
  path: 'upload',
  loadChildren: () => import('./features/upload/upload.routes'),
  canActivate: [authGuard],
},
{ path: '', pathMatch: 'full', redirectTo: 'history' },
```

- `loadChildren` charge chaque écran en *lazy loading* : un chunk par fonctionnalité.
- `canActivate: [authGuard]` protège `/upload` et `/history`. `/login`, `/register` et `/d/:token` restent publics (le destinataire d'un lien n'a pas de compte).
- Le composant ne parle jamais HTTP directement : il passe par un service de `core/` (`AuthService`, `FileService`, `DownloadService`) qui construit l'URL et type la réponse.

Le garde ([auth.guard.ts](#)) redirige vers `/login?redirect=<url demandée>` si le JWT (JSON Web Token) est absent ou expiré. Ce contrôle est purement **ergonomique** : il lit `exp` sans vérifier la signature. La vraie barrière est côté back.

Côté back : endpoint → contrôleur → service

Chaque contrôleur est un `@RestController` fin, qui délègue au service métier :

```
@RestController
@RequestMapping("/api/files")
public class FileController {
    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    @ResponseStatus(HttpStatus.CREATED)
    public UploadResponse upload(@RequestParam("file") MultipartFile file, ...,
        @AuthenticationPrincipal Jwt jwt) {
        return fileService.upload(file, password, expirationDays, ownerId(jwt));
    }
}
```

- L'identité vient du **JWT** (`jwt.getSubject()` → UUID (Universally Unique Identifier) du propriétaire), jamais d'un paramètre fourni par le client.
- Le contrôleur ne contient pas de règle métier : validation de type, taille, expiration sont dans `FileService`.
- Les endpoints publics sont listés explicitement dans `SecurityConfig` (chaîne `@Order(1)`, `permitAll`) ; tout le reste tombe dans la chaîne `@Order(2)` : `anyRequest().authenticated()`. Par défaut, un nouvel endpoint est donc **protégé**.

Les DTO : le contrat entre les deux côtés

Un DTO est l'objet qui traverse la frontière HTTP, distinct de l'entité JPA (`StoredFile`, `User`). Le back les écrit en `record` Java, le front en `interface` TypeScript. **Rien ne les synchronise automatiquement** : c'est une correspondance de noms de champs, tenue à la main.

Sens	Back (Java)	Front (TypeScript)
Entrée inscription	<code>RegisterRequest(email, password)</code> avec <code>@Valid</code>	corps de <code>AuthService.register</code>
Sortie auth	<code>TokenResponse(accessToken, tokenType, expiresIn)</code>	<code>TokenResponse</code> (<code>auth.model.ts</code>)
Sortie upload	<code>UploadResponse(downloadUrl, token, name, sizeBytes, expiresAt)</code>	<code>UploadResponse</code> (<code>file.model.ts</code>)
Sortie historique	<code>FileSummaryResponse</code> (7 champs)	<code>FileSummary</code>
Sortie lien public	<code>FileMetadataResponse(name, sizeBytes, expiresAt, passwordProtected)</code>	<code>FileMetadata</code> (<code>download.model.ts</code>)
Erreur	<code>ErrorResponse(timestamp, status, error, code, message, path, debug)</code>	<code>api-error.ts</code>

Piège de vocabulaire : `DESIGN.md` écrit `FileMetadata` et `FileSummary[]` (noms **front**) ; les classes Java s'appellent `FileMetadataResponse` et `FileSummaryResponse`. Même contrat, deux noms.

Exemple complet : US02 — télécharger via un lien

1. Route `/d/:token` (publique) → composant `features/download`.
2. `DownloadService.metadata(token)` fait `GET /api/d/{token}` → `DownloadController.metadata` → `FileMetadataResponse` : le front affiche nom, taille, expiration, et un champ mot de passe si `passwordProtected` est vrai.
3. `DownloadService.download(token, password)` fait `POST /api/d/{token}` avec `{ password }` (`DownloadRequest`) → flux binaire avec `Content-Disposition: attachment` et `X-Content-Type-Options: nosniff`.
4. Erreurs : lien expiré → `410` (`ErrorCode.EXPIRED`), mauvais mot de passe → `403` (`FORBIDDEN`). Le service passe `skipErrorNotification()` pour que l'écran affiche l'erreur dans sa carte plutôt que par le toast global.

`GET` lit les métadonnées, `POST` télécharge : le mot de passe voyage dans le **corps** et non dans l'URL, donc hors des journaux d'accès et de l'historique du navigateur.

Dans DataShare

- La carte : `DESIGN.md §1`.
- Routes et garde : `app.routes.common.ts:5-20`, `auth.guard.ts`.
- Jeton ajouté aux requêtes sauf `/auth/*` et `/d/*` : `jwt.interceptor.ts:14-26`.
- Services front : `file.service.ts`, `download.service.ts`, `auth.service.ts`.
- Contrôleurs : `AuthController.java`, `FileController.java`, `DownloadController.java`, `CurrentUserController.java`.
- DTO back : `file/dto/` et `auth/dto/`.
- Sécurité des endpoints : `SecurityConfig.java`.
- Écarts actuels : US07 (upload anonyme, hors MVP (Minimum Viable Product)) : `/upload` garde son `authGuard` et `POST /api/files` exige un JWT ; US08 (tags) non traitée ; US10 : la purge (`ExpiredFilePurger`) est testée mais aucun `@Scheduled` ne la déclenche encore.

À retenir

1. La carte de `DESIGN.md` est la table US → route → composants → endpoint → DTO : une US est terminée quand toute sa ligne existe des deux côtés.
2. Une US peut réutiliser les routes et endpoints d'une autre (US09 sur US01/US02) en ajoutant un champ optionnel plutôt qu'un nouvel endpoint.
3. Le front redirige (`authGuard`, ergonomie), le back protège (chaîne de sécurité, JWT vérifié) : le garde côté client ne remplace jamais le contrôle serveur.
4. Le DTO est le contrat : `record` Java d'un côté, `interface` TypeScript de l'autre, tenus en phase à la main ; `DESIGN.md` cite les noms front, le code Java ajoute le suffixe `Response`.
5. Le statut de la carte doit rester honnête : US07 et US08 non traitées, US10 sans déclencheur planifié.